

Skill Ramp-Up Plan – Part 4 (vue3)

Vue 3 Training Plan

Block 0 – Opening & Navigation (DevTools basics)

Skills

- I can open DevTools quickly with F12 or Ctrl+Shift+I / Cmd+Opt+I.
- I can select an element on the page with the “Inspect” icon (Ctrl+Shift+C).
- I can inspect and live-edit HTML and CSS inside the Elements panel.
- I can read computed styles to understand margins, padding, colors, and fonts.
- I can enable/disable CSS rules or add new ones directly.
- I can visualize the box model (margin, padding, border) graphically.
- I can use rulers, flexbox and grid overlays to see positioning and layout zones.
- I can simulate :hover or :active states.
- I can use the Console to read JavaScript errors and warnings.
- I can run small snippets (e.g. `document.querySelector`) and check variables or events.

Resources

- [Chrome DevTools overview](#)
- [Inspect CSS Grid & Flexbox](#)
- [DevTools Console](#)

Definition of Done (Exercise)

Open DevTools on a webpage, inspect a DOM element, toggle a CSS property on and off, visualize the box model, activate a flexbox overlay, and log a message in the console.

Block 1 – Creating a Vue Instance

Skills

- I can create a new Vue app with `createApp` .
- I can mount my app to a DOM element with `.mount( '#app' )` .
- I can add a `data` property and display it in the DOM.

Resources

- [Vue 3 Docs – Getting Started](#)
- [createApp API](#)

Definition of Done (Exercise)

Create an `index.html` with `<div id="app"></div>`.

In `main.js`, mount a Vue app displaying `Hello Vue!`.

Block 2 – Template Syntax & Interpolation

Skills

- I can use `{{ }}` to display variables.
- I can bind attributes with `v-bind`.

Resources

- [Template Syntax](#)

Definition of Done (Exercise)

Create a variable `title: 'My Vue App'` in `data()`.

Display it in an `<h1>{{ title }}</h1>` and bind it to an image `:alt="title"`.

Block 3 – Directives Basics

Skills

- I can conditionally render elements with `v-if`, `v-else`, `v-show`.
- I can loop with `v-for`.

Resources

- [Conditional Rendering](#)
- [List Rendering](#)

Definition of Done (Exercise)

Create a `todos` array in `data()`.

Render each todo with `v-for`.

If empty, show `<p>No tasks yet</p>` with `v-if`.

Block 4 – Events Handling

Skills

- I can handle user actions with `@click`, `@input`.

Resources

- [Event Handling](#)

Definition of Done (Exercise)

Create a button that increments a `count` variable on click and displays `You clicked X times`.

Block 5 – Data, Reactive & Ref

Skills

- I can use `ref` for primitive values.
- I can use `reactive` for objects.

Resources

- [Reactivity Fundamentals](#)

Definition of Done (Exercise)

Create an input bound with `v-model` to a `ref` variable `name`.

Display `"Hello, {{ name }}"`.

Add a `reactive` object `{ firstName, lastName }` and display `{{ firstName }} {{ lastName }}`.

Block 6 – Methods & Computed Properties

Skills

- I can define functions in `methods`.
- I can derive values with `computed`.

Resources

- [Computed Properties](#)

Definition of Done (Exercise)

Create two inputs for `firstName` and `lastName`.

Define a computed property `fullName`.

Display `Hello {{ fullName }}`.

Block 7 – Watchers

Skills

- I can react to data changes with `watch`.

Resources

- [Watchers](#)

Definition of Done (Exercise)

Create an input `search` with `v-model`.

Watch `search` and log `Searching for: <value>` whenever it changes.

Block 8 – Two-way Binding (v-model)

Skills

- I can bind form inputs with `v-model`.

Resources

- [Form Input Bindings](#)

Definition of Done (Exercise)

Build a login form with `v-model` bound to `email` and `password`.

Display the values live under the form.

Block 9 – Props (Parent → Child)

Skills

- I can pass data from parent to child with props.

Resources

- [Props](#)

Definition of Done (Exercise)

Create a `UserCard.vue` component with props `name` and `age` .
From the parent, pass values and display “John – 25 years old”.

Block 10 – Events (Child → Parent)

Skills

- I can use `$emit` to send data to parent.

Resources

- [Component Events](#)

Definition of Done (Exercise)

Create a `CounterButton.vue` child with a button that emits `increment` .
In parent, listen with `@increment` and update a total counter.

Block 11 – Slots Basics

Skills

- I can use `<slot>` for flexible content.

Resources

- [Slots](#)

Definition of Done (Exercise)

Create a `Card.vue` component with a `<slot>` .
Use it with:

```
<Card>
  <h2>Title</h2>
```

```
<p>Some description</p>
</Card>
```

Block 12 – Dynamic Classes & Styles

Skills

- I can toggle CSS classes and styles dynamically.

Resources

- [Class and Style Bindings](#)

Definition of Done (Exercise)

Create a button with `:class="{ active: isActive }"`.

Toggle `isActive` with `@click`.

When true, the button turns green.

Block 13 – Lifecycle Hooks

Skills

- I can use `onMounted`, `onUpdated`, `onUnmounted`.

Resources

- [Lifecycle Hooks](#)

Definition of Done (Exercise)

Create a component that logs:

- “Mounted” in `onMounted`
 - “Updated” in `onUpdated`
 - “Destroyed” in `onUnmounted`
- Toggle it with `v-if` to test.

Block 14 – Mini-project: Todo App

Skills

- I can combine Vue concepts (data, events, props, emit, v-model, slots, lifecycle).
- I can manage a list of items (create, display, update, delete).
- I can pass data between parent and child.
- I can apply conditional rendering and dynamic classes.

Resources

- [Vue 3 Guide – Essentials](#)
- [Component Basics](#)
- [TodoMVC with Vue 3 \(Example\)](#)

Definition of Done (Exercise)

Build a **Todo App** with the following features:

1. Add task

- Input with `v-model="newTask"`.
- Button adds task to `tasks` array.

2. Display tasks

- Use `v-for` to render tasks.
- Each task shows text + checkbox.

3. Mark as done

- Checkbox toggles `done` boolean.
- Apply strike-through via `:class="{ completed: task.done }"`.

4. Delete task

- Delete button removes a task.

5. Componentization

- Create `TaskItem.vue` that accepts a `task` prop.
- Emits `delete-task` and `toggle-task`.
- Parent listens and updates state.

6. Bonus

- Use a slot in `TaskItem.vue` for custom button content.
- Log "Todo App loaded" with `onMounted`.

👉 Résultat attendu : une mini-app avec au moins 2 composants (`App.vue` , `TaskItem.vue`) permettant d'ajouter, cocher/décocher et supprimer des tâches.